

Хакатон Московского транспорта

Предиктор изменений в графике движения городского транспорта

Краткое описание

Точное соблюдение графика движения — главная метрика качества наземного пассажирского транспорта. Ваша задача: разработать ИИ-систему для раннего прогнозирования изменений в движении транспорта (за 10-15 минут до фактического наступления задержки). Система должна непрерывно анализировать потоковые телематические данные (формат NDTP), сопоставлять их с эталонным расписанием, выявлять паттерны сбоев и визуализировать проблемные участки на дашборде диспетчера. Это позволит операторам переходить от реактивного управления (когда транспорт уже опоздал) к проактивному — принимать упреждающие меры.

Проблематика

Сегодня диспетчерские службы наземного транспорта вынуждены реагировать на инциденты постфактум. Если транспортное средство (ТС) уже опоздало на остановку или сбилось с интервала, диспетчер может лишь констатировать факт и пытаться «догнать» график, что часто приводит к эффекту домино (нарушению интервалов на всем маршруте).

Отсутствие инструментов раннего предупреждения приводит к:

- Снижению качества перевозки пассажиров и росту социального напряжения.
- Нерациональному использованию подвижного состава.
- Невозможности применения упреждающих мер (например, выпуск резервного ТС, изменение схемы объезда, корректировка интервалов на светофорах).

Образ решения

Ваша задача — создать комплексное приложение, состоящее из вычислительного ML-модуля, Backend-сервиса и BI-дашборда. Система в реальном времени потребляет телеметрию, накладывает её на нитку графика, с помощью ML-моделей вычисляет вероятность отклонения в горизонте 10-15 минут, определяет паттерн/причину сбоя и выводит алерт на экран диспетчера с рекомендациями.

С датасетом можно ознакомиться по ссылке:

<https://disk.yandex.ru/d/CA6tsj4aJJ4Aaw>.

Ключевые требования

- Обеспечение горизонта прогнозирования строго 10-15 минут до наступления события.
- Разделение архитектуры на независимые модули: Backend (API, оркестрация) и ML-модуль (инференс/обучение).
- Наличие понятного и информативного интерфейса для принятия решений диспетчером.

Функциональные требования

1. Анализ и обработка потоковых данных

- Непрерывный прием и парсинг потоковых телематических данных в формате NDTP (от эмулятора и исторического датасета).
- Сопоставление телематических данных (координаты, скорость, статус дверей) с эталонным историческим и текущим расписанием.
- Очистка данных и расчет производных признаков (отклонение от графика на текущий момент, средняя скорость на сегменте, время простоя).

2. Предиктивное моделирование (ML-ядро)

- Прогнозирование вероятности задержки ТС в горизонте 10-15 минут.
- Расчет абсолютной ошибки прогнозируемого времени отклонения.
- Выявление паттернов поведения ТС, предшествующих сбою (например, аномальное снижение скорости на подходе к перекрестку, длительное время посадки).

3. Диспетчерский BI-дашборд

- Визуализация маршрутной сети с наложением текущего положения ТС.
- Цветовая индикация уровня риска на маршрутах (зеленый/желтый/красный).
- Карточка инцидента: отображение ТС с высокой вероятностью задержки, прогнозируемое время опоздания, предполагаемая причина и участок маршрута.

Общие требования (являются рекомендацией, а не жестким правилом)

- Архитектура: Решение должно содержать четко выделенные модули: Backend (web-сервис) и вычислительный модуль модели машинного обучения.
- стек и сборка: Python 3.12+, PyTorch, CatBoost. Обязательна упаковка решения в Docker-контейнеры.

- Документация: Наличие сгенерированной документации по коду (PyDoc/Sphinx) и спецификации API (OpenAPI/Swagger).

Нефункциональные требования

- Производительность: ML-модуль и Backend должны обеспечивать низкую задержку (Latency) при обработке предиктов для потока ТС, чтобы диспетчер видел картину в реальном времени.
- Надежность: Корректная обработка обрывов связи с эмулятором телематики, деградация функционала (переход на работу только по историческим данным или последнему известному состоянию) без падения сервиса.
- Масштабируемость: Архитектура ML-модуля должна допускать дообучение на новых данных и масштабирование инференса.
- Использование ресурсов: Приветствуется и поощряется использование ресурсов графического ускорителя (GPU) для обучения и/или инференса нейросетей.

Дополнительные фичи

- Реализация сложной логики маппинга координат на маршрутную сеть (Map Matching) с учетом специфики NDTP.
- Внедрение механизма "What-if" анализа на дашборде (например, оценка влияния выпуска дополнительного ТС на график).
- Оптимизация инференса модели (квантование, ONNX, TensorRT) для достижения минимального Latency.
- Использование ансамблей моделей (например, связка CatBoost для табличных признаков и PyTorch/Transformer для последовательностей телеметрии).

Критерии оценивания основного этапа

№	Критерий	Баллы
1	Точность ML-прогноза (MAE по таблице прогона сабмитов)	0-6
2	Соблюдение горизонта 10–15 минут и раннее предупреждение	0-4
3	Работающая система из 3 модулей + запуск в Docker	0-6
4	Диспетчерский BI-дашборд	0-6
5	Производительность и надёжность	0-4

Детализация критериев

Критерий 1. Точность ML-прогноза (0-6)

скор из раздела Data Science	Баллы
≥ 0.70	6
0.60 – 0.69	5
0.50 – 0.59	4
0.40 – 0.49 (уровень baseline)	3
0.25 – 0.39	2
0.10 – 0.24	1
< 0.10 или нет валидного сабмита	0

Критерий 2. Соблюдение горизонта 10–15 минут и раннее предупреждение (0-4)

- Прогноз/алерт стабильно формируется в окне **T+10...15 минут** до события — основа балла.
- Нет «прогнозов задним числом» (алерт после наступления сбоя); в идеале указывается предполагаемое время опоздания и причина.
- Горизонт соблюдается на потоке, а не только на офлайн-выгрузке.

Критерий 3. Работающая система из 3 модулей + запуск в Docker (0-6)

- Решение упаковано в **Docker-контейнеры** и поднимается по одной инструкции.
- Чётко выделены **ML-ядро** (инференс/обучение) и **Backend** (API, оркестрация) — не монолит; API работает (Swagger отвечает на пробный запрос).
- Backend принимает и **парсит поток телеметрии NDTP**, сопоставляет с расписанием и считает производные признаки (текущее отклонение, средняя скорость на сегменте, время простоя).
- Связь модулей рабочая end-to-end: поток → прогноз → дашборд.

Критерий 4. Диспетчерский VI-дашборд (0-6)

- **Понятность за ~5 секунд**: диспетчер видит проблемные ТС/участки без инструкции.
- Карта маршрутной сети с **текущим положением ТС** и **цветовой индикацией риска** (зелёный/жёлтый/красный).
- **Карточка инцидента**: ТС с высокой вероятностью задержки, прогнозируемое опоздание, предполагаемая причина, участок маршрута.
- Данные обновляются в реальном времени вслед за потоком.

Критерий 5. Производительность и надёжность (0-4)

- **Latency инференса** низкая (ориентир < 1–2 с на поток ТС); честная оценка с подтверждением метриками засчитывается без полного нагрузочного теста.
- **Пропускная способность**: поток ТС обрабатывается без накопления очередей.
- **Деградация при обрывах связи**: сервис не падает, переходит на исторические данные / последнее состояние и восстанавливается после реконнекта.
- **Холодный старт** Docker-контейнеров предсказуем. Балл снижается при отсутствии подтверждений, но не обнуляется при корректной архитектуре.

Загрузка решения

Решение загружается на платформе хакатона в двух обязательных версиях:

1. Загрузка csv-файла в разделе **«Data Science»**.

Лимиты загрузок на каждую команду в день:

- **общее кол-во попыток – 36** (все попытки, включая ошибочные);
- **кол-во успешных попыток – 24** (попытки со скором 0 и выше).

Загружать решение в данном разделе можно **до 27 сентября, 23:59 МСК**.

В зачет пойдет лучший результат, полученный за всё время.

2. Заполнение полей формы (ссылка/ввод текста) в разделе **«Загрузка решения»**.

Обновлять решение в данном разделе можно вплоть до дедлайна регистрации – **27 сентября, 23:59 МСК**.

Сохранится и пойдет на проверку **заключительная версия**.

Обязательные артефакты на сдачу

(заполнение полей формы на платформе хакатона с указанием ссылок на материалы)

1. Сабмит (.csv) с прогнозами по validate для авто-проверки MAE (файл, который загружается участниками для расчета значения метрики в разделе «Data Science»).
2. Ссылка на рабочую систему из 3 модулей (ML-ядро + Backend + BI-дашборд) в Docker, запускаемая по README.
3. Ссылка на инструкцию для жюри: как подать поток (эмулятор NDTP / исторический датасет), где увидеть прогнозы и алерты, как открыть дашборд и метрики.
4. Ссылка на документацию (PyDoc/Sphinx по коду и OpenAPI/Swagger по API).
5. Описание сведений о производительности и список реализованных дополнительных возможностей.

Оценка полуфинальных питчей

№	Критерий	Баллы
1	Оценка выступления	0-10

Итоговый максимум по этапам 1-2 — 36 баллов.

Оценивается само выступление, уровень и глубина погружения в тему и корректность ответов на вопросы жюри.

- Качество выступления и демо (0-3).
- Уровень и глубина погружения в тему (0-3).
- Корректность ответов на вопросы жюри (0-4).